Where to Stop:
Tile-Adaptive Early Exit in a Learned Image Decoder

Anonymous CVPR submission · Paper ID ****

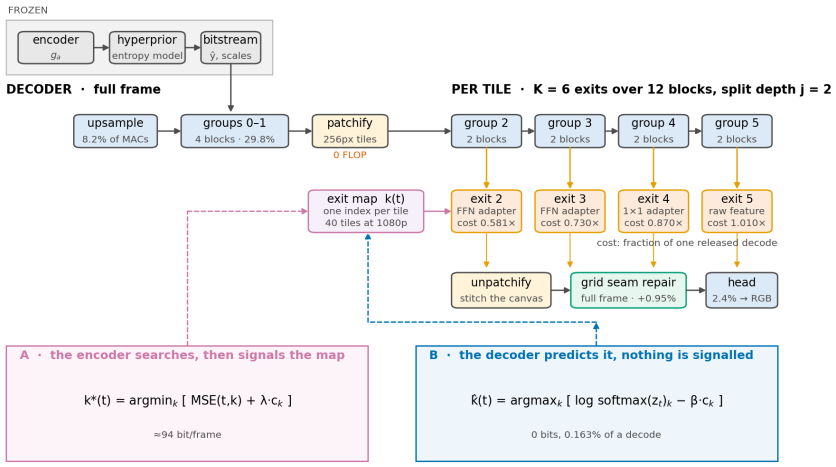

Figure 1. FLEX-UF end to end. Grey is frozen and never touched; blue is inherited from DCVC-UF and fine-tuned; orange and green are new. The first j block groups run over the whole frame, the rest run per tile on a shrinking active set, and the exit map that drives the shrinkage comes either from an encoder-side search (A) or from a decoder-side predictor (B).

Abstract

A learned image decoder spends the same computation on every frame regardless of what the frame contains. We add an early-exit ladder to the intra decoder of DCVC-UF, cut each frame into tiles, and let every tile leave the trunk at whichever of K exits is deep enough for it. The encoder is frozen and the coded payload is unchanged, so the method deploys against existing bitstreams. On the full 53-sequence common test set a 0.1 dB budget buys 32.3% of the decoder's multiply-accumulates at the lowest rate and 16.8% at the highest, for a BD-Rate cost of 0.88%.

Four findings are worth more than the headline. **(i)** A quality budget only buys compute inside a bounded window: below a floor set by tiling no allocation is feasible, above a saturation point every tile already sits on the cheapest rung, and the working budget uses 45% of that window at the lowest rate and 9% at the highest. Both ends are in closed form and we verify seven structural propositions numerically. **(ii)** Tiling is expensive and its cost is governed by depth, not area: the seam grows as b^2 in the number of per-tile convolutions, where the corrupted-area fraction usually quoted is wrong by 160–264%. **(iii)** The exact remedy — give each convolution its real neighbour — is bit-identical at uniform depth and *destroys* the allocation under routing, because routing is the deliberate violation of the condition that makes it exact. **(iv)** Operations are an *optimistic* bound on this method and the optimism grows with the saving: 29.1% of wall-clock arrives against a 35.3% arithmetic prediction at the lowest rate, 15.6% against 20.1% at the highest. Sorting the tiles once by exit depth is bit-identical and recovers a fifth of the shortfall.

Signalling the exit map costs 79 bits per frame; predicting it at the decoder costs none and gives up 1.7–13.3 points. The two are ends of one scale rather than two designs: overriding the worst fifth of tiles recovers 42–83% of that gap for 44 bits, and how much any fraction can recover is bounded above by the Lorenz curve of the per-tile regret.

We also find that the method is governed by tile count far more than by content — three 1080p classes of very different material agree within three points while a 416×240 class saves a third as much — and that an exit map transfers across frames almost perfectly but not across rates.

1. Introduction

Learned codecs are compared on rate and distortion, with complexity reported as a single number: so many GMAC per frame, so many milliseconds. That number is a constant. A learned decoder runs the same graph on a page of text and on a cloudless sky, and the sky is not harder.

Classification networks abandoned this a decade ago. Early-exit architectures attach classifiers at intermediate depths and stop once the prediction is confident [3, 15, 20]. Super-resolution adopted the idea spatially: ClassSR [12] routes patches to networks of different capacity, APE [1] exits patches at different depths. Learned compression went another way — slimmable autoencoders [22, 23] give one model several complexity levels, but the level is chosen *per stream*, not per region, and switching it changes the bitstream.

We ask the spatial-adaptivity question inside a learned decoder under a constraint that makes the answer deployable: **the encoder is frozen and the coded payload is unchanged.** That rules out retraining the analysis transform, changing the entropy model or altering the latent, and leaves exactly one place to spend adaptivity — the synthesis transform.

FLEX-UF is an exit ladder over the twelve residual blocks of the DCVC-UF intra decoder. The first j blocks run over the whole frame; the rest run per tile on a shrinking set, and that shrinkage is the saving. Each exit reaches the shared head through a small pointwise adapter, zero-initialised so that at step zero the deepest exit reproduces the released decoder bit-exactly.

Making it work depended less on the ladder than on two things that have little to do with early exit as usually studied.

Tiling has a price, and it is large.

Spatial adaptivity needs tiles, and the moment a frame becomes tiles every 3×3 convolution at a border reads invented values. The error is 0.677 dB at high rate under the stock padding rule — several times the budget, paid before any tile has saved an operation. Section 4 treats padding as an *estimator* of the unseen neighbour and measures four; the ordering is not the obvious one.

A quality budget has a working range.

Because tiling costs something even when nothing exits early there is a *floor*: budgets below it admit no allocation. Because the ladder has a shallowest rung there is a *saturation* point above which more quality buys nothing. The working 0.1 dB budget uses 45% of that window at the lowest rate and 9% at the highest.

Contributions.

(i) A tile-adaptive early-exit ladder for a learned image decoder that leaves the encoder and the coded payload untouched, saving 32.3% to 16.8% of decoder MACs for 0.1 dB on the full CTC set at a BD-Rate cost of 0.88%. **(ii)** The floor/saturation characterisation of a quality budget, both ends in closed form, with seven structural propositions verified numerically and a measurement of what the Lagrangian's convex-hull restriction costs (at most 0.05 saving points). **(iii)** A quantitative account of the tiling penalty: the b^2 law and the failure of the area law, the measured ordering of four border estimators, and two remedies rejected on their own numbers. **(iv)** Two allocation regimes with bits and compute charged on both sides, plus a free zero-parameter baseline that recovers 75% of the oracle's advantage over chance — a control we suggest any adaptive-inference paper should report. **(v)** A wall-clock result: operations over-predict the saving by about a fifth, and a bit-identical reordering of the per-tile loop recovers a fifth of that — 29.1% realised against 35.3% predicted. **(vi)** Two properties of the allocation that bear on deployment — it is governed by tile count rather than content, and it transfers across frames but not across rates.

2. Related work

Early exit.

BranchyNet [20] and MSDNet [3] established intermediate classifiers with a confidence rule; SDN [15] framed it as mitigating overthinking. Joint training of all exits follows Scardapane et al. [18]; distillation between exits follows Phuong and Lampert [17], with the caveat from [19] that too large a student-teacher gap hurts the shallowest exits — which is why ours is between *adjacent* exits. All of this exits on a confidence signal computed from the network's own output. A decoder has no such signal: the quantity that would decide is the error against a source it cannot see.

Spatially adaptive inference.

ClassSR [12] sorts patches into easy/medium/hard; APE [1] exits patches at different depths; Glance-and-Focus [8] spends resolution adaptively. These share our per-region premise and share the tiling problem, though the cost of tile borders is rarely quantified. The closest prior work on the border itself is the per-channel AR(1) padding of Kaseva et al. [11], which we implement and evaluate.

Method	What varies	Granularity	New bitstream
SlimCAE~citeslimcae	width	per stream	yes
Slimmable video~citeslimvc	width	per stream	yes
EVC~citeevc	mask / pruning	per model	yes
Spatial competition~cites patialcompetition	which codec	per region	yes
DCVC-RT~citedcvcrt	architecture	fixed	yes
FLEX-UF	**decoder depth**	**per region**	**no**

Table 1. Where this sits. Complexity control in learned compression varies the model; we vary how much of a fixed model runs where. The last column is what makes the difference operational: every other row requires a decoder that matches the encoder that produced the stream.

Complexity control in learned compression.

SlimCAE [22] and slimmable video codecs [23] expose several widths of one model; the choice is per stream and changes the encoder, so the bitstream is not interchangeable. DCVC-FM [13] and DCVC-UF [14] reduce cost architecturally, for every frame equally. Rate–distortion–complexity has since become an explicit third axis: Gao et al. [35] tune spatial context usage to trade decode cost against rate, Ho et al. [36] survey where conditional residual coding sits on that surface, and Zhang and Gao [37] route *whole frames* to one of several jointly-trained coding paths. All of these vary the model, and all change what the encoder emits. Our axis is orthogonal: model fixed, bitstream fixed, only *how much of the decoder runs where* varies.

The closest neighbour.

Blard et al. [27] also partition an image into regions, also choose per region by a rate–distortion cost computed at the encoder, and also transmit a mode map — the skeleton of our configuration A. Two things differ. Their regions choose among several *separately trained* codecs, so the decoder must hold all of them and its complexity is that of one codec regardless of the choice; the map buys rate, not compute. Ours choose a prefix length of a single trunk, so the weights are shared by construction and the map buys compute at fixed rate. And because their alternatives are unrelated networks, no decoder-side predictor could stand in for the encoder's search, whereas the nesting that makes our exits prefixes of one another is exactly what makes configuration B possible at all.

Signalling versus prediction.

That a decoder is *told* a mode decision rather than inferring it is the norm in standardised video coding: HEVC [9] and VVC [21] transmit partitioning, prediction mode and transform tree. We evaluate both and treat the signalled variant as the conventional design. The nearest neighbour in compression is spatial competition [27], which selects among several codecs per region and signals a mode map: the structure of the side information is ours, the axis is not — they select which network at fixed complexity for a rate gain, we select how much of one network at fixed rate for compute.

Deferring to an oracle under a budget.

Configuration C, where a predictor decides most cases and a small budget of the hardest ones is handed to something exact, is the shape of selective prediction [38] and learning to defer [39, 40], and of the budgeted variant of the latter [41]. Two things differ and both make our case easier. The expert here is the encoder's own search, so it is exact, always available, and costs nothing at test time — what is scarce is not the expert's attention but the *bits* needed to say what it decided. And the selection rule is not learned: because the objective is separable over tiles, the optimal set of size s at a fixed multiplier is exactly the s largest regrets, which the encoder can compute. The open question in that literature — who to defer, and how to learn it — has a closed form here, and what remains is the question we measure, which is how concentrated

the regret is.

Allocating a budget over units.

The construction we use is not new and should not be presented as such. Shoham and Gersho [28] showed that for a finite set of per-unit operating points a Lagrangian sweep decouples the allocation across units and traces exactly the lower convex hull of the achievable set; Ortega and Ramchandran [29] made it standard practice in image and video coding. What we add is the structure this particular operating set has — a floor below which no allocation is feasible, a saturation point above which none improves, and a measurement of what the convex-hull restriction costs. The same relaxation has resurfaced for test-time compute in language models [30], with per-instance decoupling and a binary search on the multiplier: the identical structure in a domain with no rate axis.

How much is there to gain?

Bounding what adaptive inference could achieve is itself a line of work. Hasan et al. [34] derive an oracle bound on efficiency at fixed accuracy from per-model resource and accuracy, and report 43–121× on ImageNet and 7–81× on HellaSwag. Their bound has a ceiling and no floor, because the largest model in their family attains the reference accuracy by definition. Spatial adaptivity introduces one: cutting a frame into tiles costs quality even when every tile runs to full depth, so the reference is unreachable at *any* compute and the feasible set of budgets is an interval rather than a ray. That interval, and both of its ends, is what Section 5.4 characterises.

Tile boundaries.

Every method that processes an image in independently-computed tiles meets the same artefact, and the remedies in the literature are overlap and averaging, local padding from neighbouring patches [31], training with overlaps [32], and fitted extrapolation [11]. Local padding is closest to the exact remedy of Section 4.4, and the difference is accounting: it pads every convolutional layer and does not report the cost, while we pad only the 0.29% of each block that has any spatial extent — which is what makes the exact fix cost 0.032% of the decode rather than a multiplier on all of it.

Where the time goes.

DCVC-RT [33] argues that operational rather than computational complexity is the speed bottleneck for neural codecs, evidenced by channel reductions that yield linear rather than quadratic speedups. Section 5.8 is an instance of that claim inside one loop: 35.3% of operations removed buys 29.1% of time, and 1.2 points of the difference come back from reordering the loop with the arithmetic untouched.

3. Method

3.1 Where the computation is

The DCVC-UF intra decoder is one upsampling block, twelve DepthConvBlocks and a head, costing 453.5 GMAC per 1080p frame. The twelve blocks are 89.4% of it, which is why the ladder is built across them. Inside one block at C=384, the only operator with any spatial extent is a 3×3 depthwise costing 9C against the block's $8C^2+9C$ — **0.29%.** That number recurs throughout: it is why tile borders damage the picture, why our adapters are pointwise on purpose, and why the exact remedy of Section 4.4 is affordable.

3.2 The exit ladder

With K exits over the twelve blocks and split depth j, groups 0..j–1 run full frame for every tile and groups j..K–1 run per tile. A tile assigned exit k runs groups j..k and leaves. Writing c_k for the cost of exit k in units of one released decode, a frame with exit map k costs the mean of c over its tiles.

Two consequences are easy to state wrongly. Exits shallower than j are indistinguishable, so the usable ladder has K–j distinct costs, and the *architectural ceiling* is S_max = 100(1–c_j)%, attained when every tile takes exit j. For the shipped setting (K=6, j=2, 256 px tiles) S_max = 41.9%. Section 5.4 shows this bound is *reached* at low rate, which makes it an operating point rather than an asymptote.

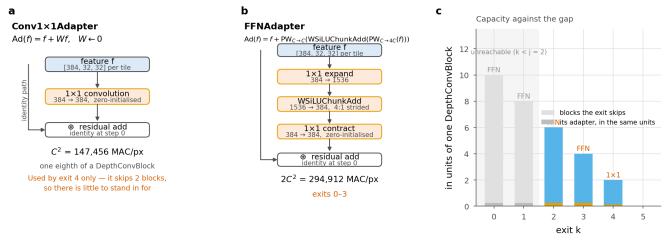

Figure 2. Inside an exit adapter. Both kinds are entirely pointwise, so no adapter adds receptive field and none contributes any tile-boundary penalty. Capacity is matched to the number of blocks the exit skips, and charged for: an exit-2 tile saves six blocks minus 0.25, not six.

3.3 Exit adapters

An early exit hands the shared head a feature the head was not fitted to; the adapter is the correction. We use a residual 1×1, Ad(f) = f + Wf with W zero-initialised, for exits that skip little, and the pointwise expand/activate/contract pair of the block's own FFN for exits that skip four blocks or more. Both are *pointwise by design*: a 3×3 inside an adapter would add seam damage precisely at the tiles that took an early exit, the ones least able to afford it. Zero-initialising the last layer makes every adapter the identity at step zero, so the deepest exit is bit-exactly the released decoder before training begins.

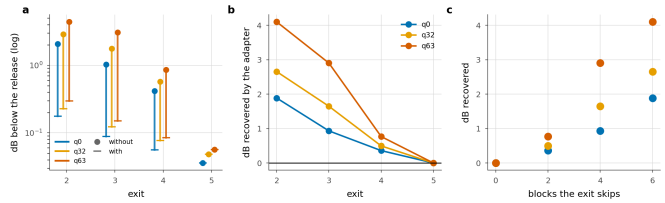

Figure 3. What the adapters are worth. a, each exit with and without its adapter. **b**, the dB the adapter recovers. **c**, the same against the number of blocks the exit skips — the design rationale, measured.

	q0		q32		q63	
Exit	with	without	with	without	with	without
2	0.177	2.065	0.228	2.884	0.297	4.398
3	0.089	1.023	0.123	1.772	0.151	3.054
4	0.056	0.416	0.077	0.574	0.084	0.853
5 †	0.036	0.036	0.048	0.048	0.056	0.056

Table 2. What the adapters are worth. dB below the release with every tile at that exit, with the trained adapters and with each set back to the identity it was initialised to. † the deepest exit has no adapter by construction and is the control.

Zeroing them measures what they learned, since the identity is exactly what they were initialised to. Without adapters the shallowest exit costs 4.40 dB at q63 — forty-four times the budget — and the adapter buys 4.10 dB back. The gain grows with rate and with the number of blocks the exit skips, which is the design rationale measured rather than argued, and the deepest exit moves by exactly zero. The adapters are not a refinement of the ladder; without them it has no usable rung.

3.4 Allocation

Given per-tile distortions D(t,k) and costs c_k, the allocation minimising distortion at a compute budget is the Lagrangian **k*(t) = argmin_k [D(t,k) + λ·c_k]**, with λ bisected so the frame lands on the budget. Both

quantities are available *to the encoder*, which holds the source. This is configuration **A**: the encoder searches and transmits the map, which entropy-codes to ~79 bits per 1080p frame, 0.021% of a typical bitrate. It costs the encoder about one extra decode and the decoder nothing.

The decoder cannot evaluate it: D(t,k) is the error against a source it never sees. This is not a hard estimation problem, it is a missing variable, and no architecture removes it. Configuration **B** therefore predicts: a 144K-parameter head reads the stem feature, the decoded latent, the entropy model's scales and the quality index, and decides **k(t) = argmax_k [log softmax(z_t)_k − β·c_k]**, with β bisected as λ is. Nothing is added to the file. The head is trained against a *frozen* decoder with a cost-sensitive cross-entropy to the oracle's choice, each tile weighted by the regret of choosing wrongly, plus loss-free load balancing [16] biased toward the oracle's own exit distribution rather than toward uniform.

3.5 Training

All exits are decoded every step and the objective is L = L_RD + w_a·L_anchor + w_d·L_distill. L_RD is the released rate-distortion loss with MSE averaged over exits. L_anchor pins the deepest exit to the released decoder, so the reference every saving is quoted against cannot drift underneath the measurement. L_distill supervises adapters in *feature* space, exit k imitating exit k+1 — feature space because the head is a fixed map from feature to RGB and matching the deeper feature is the stronger constraint, 384 dense channels of target instead of 3; adjacent rather than deepest for the reason in [19].

Critically, the adapters are trained *through the tiled decode path they are deployed in*. Training them full-frame and tiling only at inference loses 0.14–0.24 dB; training through the deployed path gains 0.51–0.90 dB. The sign of the effect flips.

4. The price of tiles

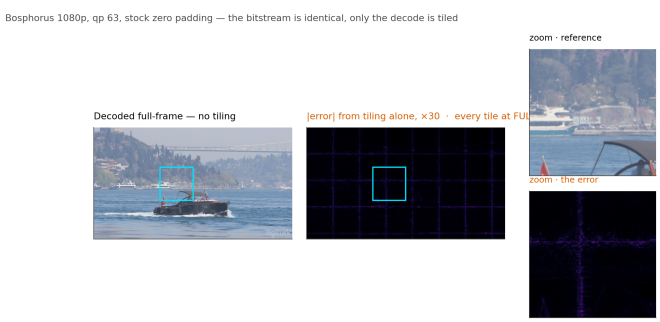

Figure 4. The artefact, before anything is done about it. One frame decoded twice from the *same* bitstream with the same weights, every tile at full depth, stock zero padding — no early exit anywhere. The only difference is that the right-hand decode was tiled. The error map is the tile lattice and nothing else.

A 3×3 depthwise computes a weighted sum over its neighbours. Decoded full frame those neighbours exist; decoded per tile they do not, and the kernel is handed whatever the padding rule invents. Each convolution extends the contaminated region by one ring, so with b per-tile blocks on a tile of side F the fraction of the tile within reach of an invented value is $1 - ((F-2b)/F)^2$, which is 0.750 at the shipped F=32, b=8: not a thin border but a structured error across most of the tile.

That fraction says which pixels are affected, not how badly, and it is not what governs the penalty. Sweeping the split depth sweeps b from 12 to 0, with b=0 as an exact zero-seam control, and the measured seam follows a power law: **seam** $\propto \mathbf{b}^{\alpha}$ with α = 2.38, 2.22 and 1.93 at q0, q32 and q63, r = 0.98–0.995 in log-log. Fitted with one free scale the area fraction is wrong by 160–264% where the power law is wrong by 12–22%. The exponent near two has a reading: the count of contaminated pixels grows like perimeter times depth, and the error accumulated in each grows with how many convolutions reached it. The area law saturates once every pixel is touched; the penalty does not.

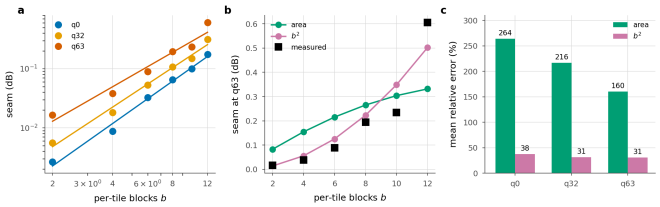

Figure 5. The seam against per-tile depth. Sweeping the split depth sweeps b; b=0 is an exact control and measures 0.0000 dB at all three rates. **a**, the measurement with the fitted power law. **b**, both models against q63 with one free scale each. **c**, mean relative error. The area fraction saturates once every pixel is contaminated; the penalty does not.

4.1 Padding is an estimator

The useful way to see border padding is as an *estimator* of the unseen neighbour, whose error is the seam. Table 1 measures four, with early exit switched off so tiling is the only difference from a full-frame decode.

Border estimator	q0	q32	q63
Zeros (stock)	0.126	0.287	0.677
Replicate	**0.071**	**0.123**	**0.218**
Linear extrap.	0.198	0.286	0.384
AR(1) per channel~citearls	0.061	0.107	0.197

Table 3. Border estimators, dB below the released decoder on the same latent, 256 px tiles, full CTC. Lower is better.

Two results deserve emphasis. **A higher-order estimator is worse.** Extrapolating the local gradient past a boundary amplifies whatever noise sits on it; assuming local constancy does not. Guessing harder is not guessing better, and the gap is large: 0.384 dB against 0.218 dB at q63. **The best estimator loses on cost.** The per-channel AR(1) fit of [11] reaches 0.197 dB, about 0.02 dB better than replication, for 10.7% of decode wall-clock. Against a 0.1 dB budget and a ~24% saving that trade does not close, and we drop it.

4.2 What actually removes the seam

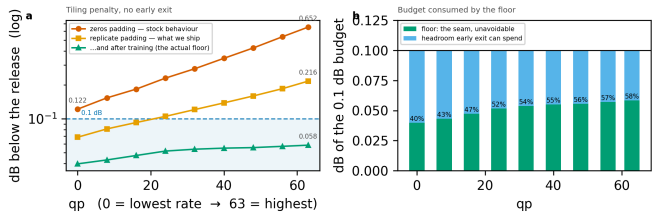

Figure 6. The tiling penalty across the whole rate range, every tile at full depth. The two steps that matter cost nothing, and the largest single factor is training the ladder with the seam present. Right: the floor is charged *inside* the quality budget and consumes a growing share of it.

Tile size is free — a tiled decode's MAC count does not depend on it at all — and the damage scales with the tile perimeter (Table 2). What it costs instead is routing granularity: 40 tiles per 1080p frame at 256 px against 160 at 128 px.

Estimator, q63	128 px	256 px	ratio
zeros	1.249	0.677	×0.54
replicate	0.372	0.218	×0.58
arls	0.332	0.197	×0.59

Table 4. Tile size, dB at q63. Doubling the side roughly halves the penalty, as the contamination law predicts, and costs no computation.

The largest factor of all is easiest to overlook. Replicate padding leaves 0.218 dB at q63 on the untrained ladder; after training the same configuration leaves 0.072 dB. More than two thirds of what the estimator could not fix is absorbed by weights learning to live with it. No module in this paper removes as much.

4.3 A learned repair module, and why we reject it

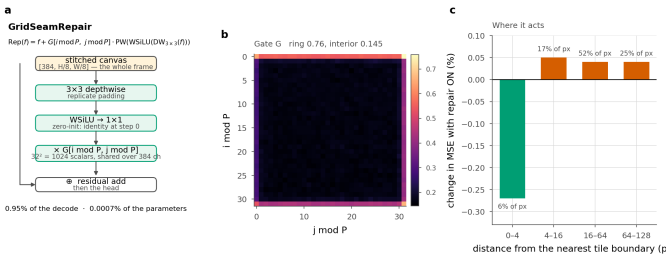

Figure 7. Grid seam repair. A correction gated by position within a tile, applied once to the stitched frame. The trained gate learns the boundary ring (0.76) but never switches off inside (0.145 over 94% of pixels), and the measured effect follows: it helps on the ring and hurts everywhere else.

Since the tile lattice is known exactly at training and inference, a repair can be *told* where to look rather than having to infer it: Rep(f) = f + G[i mod P, j mod P]·PW(WSiLU(DW3×3(f))), with G a P×P gate shared over channels, initialised at exp(−d/τ). It costs 0.95% of the decode.

It does not earn that. Splitting per-pixel error by distance from the nearest tile boundary, the module gains 0.27% in the 0–4 px band (6.2% of pixels) and loses 0.04–0.05% everywhere else. Even with a *perfect* gate — zero correction in the interior, the boundary gain unchanged — the ceiling on what it could earn is ≈0.0008 dB for 0.95% of the decode. We rejected AR(1) padding at 0.0019 dB per point of decode; this is worse by an order of magnitude. Tightening the gate cannot rescue it, because there is almost nothing left to win.

4.4 Removing the cause, and why it does not help

Because only 0.29% of each block has spatial extent, the exact fix is affordable: give the 3×3 its real neighbour from a shared canvas, for +0.032% of the decode. And it is exact — at uniform depth a coupled tiled decode is bit-identical to a full-frame one, once the comparison is not confounded by the repair module, which runs on the stitched canvas and has no full-frame counterpart. Measured: 1.13e-2 with the repair on, **exactly 0** with it off, against 6.06e-2 for replicate padding.

It also removes most of the floor. Switched on at inference the floor falls from 0.036 to 0.003 dB at q0 and from 0.056 to 0.030 at q63, i.e. 91% and 47% of the tiling penalty.

And it destroys the allocation. At the same 0.1 dB budget the saving falls from 25.9% to 4.2% at q32 and from 19.3% to 0.4% at q63.

The reason is the condition in the exactness statement. Coupling is exact when every tile is at the *same* depth, and routing is the deliberate violation of that condition. With replicate padding a tile is entirely independent of its neighbours, so the allocation may give adjacent tiles any depths it likes; coupling makes a tile depend on its neighbours, and under routing those neighbours ran a different number of blocks. A shallow tile reading a deep neighbour's activation is a configuration the weights have never seen, and that mismatch costs far more than the seam it removed. We report it because the natural reading of an exactness result — adopt the exact fix — is the wrong one here, and the tension is a property of the combination that any spatially adaptive decoder inherits.

5. Experiments

Setup.

The decoder is fine-tuned on OpenImages 512×512 crops with the encoder frozen (max|Δ| = 0 asserted every run), one λ drawn per sample from a log-spaced range covering all 64 quality indices, so a single set of weights serves the whole rate range. Evaluation is on the 53 sequences of the common test set — UVG, MCL-JCV and HEVC classes B, C, D and E — one intra frame each.

Measurement protocol.

Two details change the numbers enough to state. First, the per-tile distortion table must be built on the *deployed* decode path — one tiled decode per exit — and not by tapping the exits of a single full-frame forward pass. The latter is the natural implementation and correct for training, but with a full-frame reference on the other side of the ratio the tiling penalty cancels and the reported quality belongs to a decoder nobody ships. On our model the difference is +0.035 dB at the deepest exit and +0.008 dB at the shallowest; it grows with how many blocks ran per tile, so it cannot be corrected after the fact. Second, after the allocation is chosen we decode that *mixed* map once and report the distortion it actually produces.

Reporting conventions.

Every dB is measured against the *released* decoder's full-frame decode of the *same* latent, and our side is the deployed tiled decode, so the tiling penalty is inside every number. Savings are fractions of the released decoder's cost; our own deepest exit costs 1.0095 of it, and using that as the denominator would flatter every result by 0.6–0.8 points.

5.1 Main result

Figure 8. What the saving looks like. The same bitstream decoded by the released decoder and by ours at the 0.1 dB operating point, 30.5% fewer multiply-accumulates. The crop is the tile that gave up the most quality, chosen automatically, so this is the method's worst case on this frame rather than a flattering one.

Budget	q0	q16	q32	q48	q63	mean
0.10 dB	32.3	27.6	22.5	19.8	16.8	**23.8**
0.30 dB	41.9	41.9	41.9	40.6	38.4	**41.0**
0.50 dB	41.9	41.9	41.9	41.9	41.9	**41.9**

Table 5. Decoder MACs saved (%) against the released decoder, per quality index and budget, configuration A. The 0.3 and 0.5 dB rows sit on the architectural ceiling almost everywhere: past that point a looser budget buys nothing.

At 0.1 dB the method saves 32.3% at the lowest rate and 16.8% at the highest, averaging 23.8%, at a BD-Rate cost of 0.88% — that is, the compute is worth about the same as a 0.88% increase in bitrate. The fall with rate is not an artefact of the ladder: high-rate reconstructions carry detail the shallow exits cannot reproduce, and the floor rises with rate.

Decoder	GMAC	% of release	ms
Released DCVC-UF	454	100.0	111
FLEX-UF, all tiles deepest	458	101.0	114
FLEX-UF, 0.1 dB budget	346	76.2	78
FLEX-UF, architectural ceiling	263	58.1	–

Table 6. Decoder complexity at 1080p. Our deepest exit costs slightly more than the release because it still pays the seam-repair module; that 1.0095, not 1.0, is what every saving here is *not* divided by.

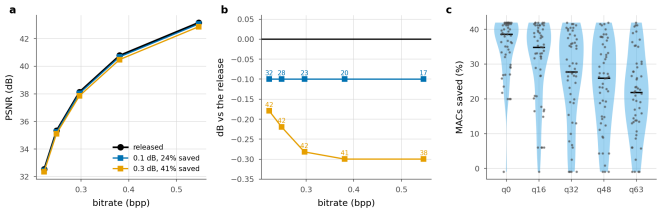

Figure 9. The plane a codec is read on, and the spread behind the mean. a, operating points against the released curve. **b**, the same with the quality axis expanded; labels are compute saved. **c**, per sequence at a matched point near 0.1 dB; one dot per sequence, bar is the median.

Panel c is the distribution the headline averages over, and it is wide. At q0 the median sequence saves 38.6% with an interquartile range of 32.0–41.5, while the worst saves −1.0% — the low-resolution sequences of Section 5.3, where two tiles leave nothing to allocate. Reporting the mean alone would hide both ends.

5.2 Is per-tile adaptivity necessary?

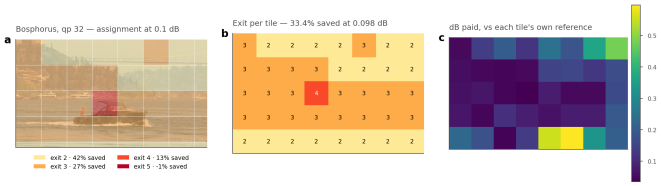

Figure 10. Where the decoder spends. (a) the assignment overlaid on the frame, (b) the exit index per tile, (c) the quality each tile gives up against its *own* full-depth reference. Water and sky leave at the shallowest rung; the boat and shoreline run deep.

q	Allocation	ΔPSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.179	41.9
	uniform, exit 3	0.093	27.0
	uniform, exit 4	0.059	13.0
	uniform, exit 5	0.036	-1.0
	random (matched mix)	0.125	32.4
	rate-ranked (free)	0.107	32.4
	oracle	**0.100**	**32.4**
16	uniform, exit 2†	0.219	41.9
	uniform, exit 3†	0.118	27.0
	uniform, exit 4	0.075	13.0
	uniform, exit 5	0.045	-1.0
	random (matched mix)	0.133	27.5
	rate-ranked (free)	0.106	27.5
	oracle	**0.100**	**27.5**
32	uniform, exit 2†	0.282	41.9
	uniform, exit 3†	0.147	27.0
	uniform, exit 4	0.090	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.141	22.5
	rate-ranked (free)	0.107	22.5
	oracle	**0.100**	**22.5**
48	uniform, exit 2†	0.333	41.9
	uniform, exit 3†	0.176	27.0
	uniform, exit 4†	0.103	13.0
	uniform, exit 5	0.062	-1.0
	random (matched mix)	0.145	19.7
	rate-ranked (free)	0.106	19.7
	oracle	**0.100**	**19.7**
63	uniform, exit 2†	0.396	41.9
	uniform, exit 3†	0.206	27.0
	uniform, exit 4†	0.116	13.0
	uniform, exit 5	0.072	-1.0
	random (matched mix)	0.141	16.7
	rate-ranked (free)	0.110	16.7
	oracle	**0.099**	**16.7**

Table 7. Adaptivity against three controls at 0.1 dB. † marks uniform depths exceeding the budget. Random and rate-ranked draw the oracle's own exit histogram — same average cost, same mix of depths — and differ only in which tile gets which depth.

The uniform rows answer the obvious alternative, and the answer sharpens with rate. At the lowest rate a static decoder reaches exit 3 within the budget and saves 27.0%, against the oracle's 32.3%. At q48 and q63 the only uniform depth that fits is the deepest, which *costs*

0.95% rather than saving anything — so there the choice is not between 17% and less, it is between 17% and nothing. Averaged over rates the best static allocation saves 10.2% where the adaptive one saves 23.8%.

The two shuffled rows separate effects that are easy to conflate. Given the oracle's histogram but assigned at random, quality collapses: what the allocation buys is not that some tiles run shallower, it is knowing *which* ones can. The rate-ranked row keeps that histogram and orders it by a signal the decoder already holds — the bits the entropy model spent on each tile. It is the cheapest conceivable router, with no parameters and no training, and it is the natural analogue of the confidence rules early-exit classifiers use.

It recovers most of the gap. At q63, random costs 0.141 dB and the oracle 0.100 dB at identical compute; rate-ranking costs 0.110 dB, i.e. 75% of the oracle's advantage over chance, at 0.68–0.79 agreement with the oracle's map. A learned router therefore has to beat a free baseline already three quarters of the way there — a comparison we would not have made without this control, and one we suggest any adaptive-inference paper should report.

5.3 Resolution, and the granularity of a tile

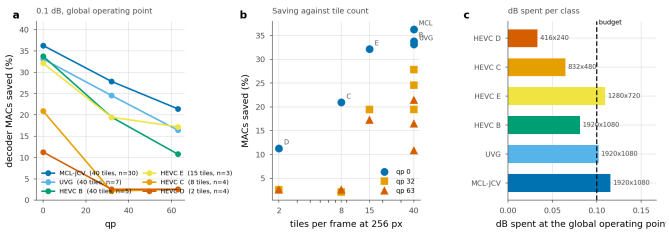

Figure 11. Saving by test class at one global operating point. λ is bisected once so the whole set lands on the budget, exactly as it would be in deployment; each class is then reported at that λ. The ordering follows tile count, not content.

Class	Resolution	Tiles	n	q0	q32	q63
MCL-JCV	1920x1080	40	30	36.3	27.9	21.4
UVG	1920x1080	40	7	33.1	24.5	16.4
HEVC_B	1920x1080	40	5	33.8	19.5	10.8
HEVC_E	1280x720	15	3	32.1	19.4	17.2
HEVC_C	832x480	8	4	20.9	2.1	2.5
HEVC_D	416x240	2	4	11.3	2.5	2.5

Table 8. Saving by class (%) at a 0.1 dB budget set globally over all 53 sequences. “Tiles” is how many 256 px tiles a frame of that resolution contains.

Completing the test set exposed a dependence the 1080p-only subset had hidden. At 1080p, where a frame is 40 tiles, the method saves 36.3% (MCL-JCV), 33.1% (UVG) and 33.8% (HEVC B) at the lowest rate — three classes of very different content within three points of each other. At 832×480 it is 8 tiles and 20.9%; at 416×240 it is 2 tiles and 11.3%. At q63 the two low-resolution classes fall to 2.5% against 21.4% for 1080p. The method is a function of tile count far more than of content.

The cause is granularity: with two tiles per frame there is almost no allocation to make and the Lagrangian degenerates toward a uniform choice. The fix is not more compute but a tile size chosen relative to the frame — the tiling penalty scales with the tile perimeter and the MAC count does not depend on tile size at all, so the trade is between seam and granularity and should be resolved per resolution. We report the single 256 px setting used throughout and note that a resolution-adaptive tile size is the obvious extension.

5.4 The working range of a quality budget

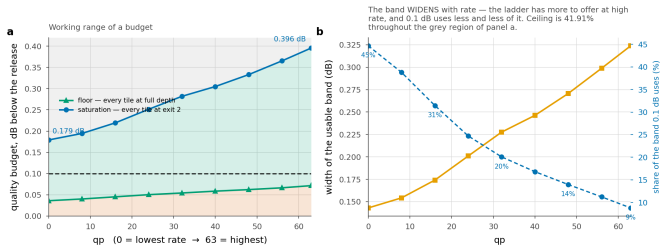

Figure 12. Three regions, and only the middle one is a design choice. Below the floor no allocation meets the budget; above saturation every tile is already on the cheapest rung. The working 0.1 dB budget uses 45% of the window at the lowest rate and 9% at the highest.

q	0	16	32	48	63
Floor D_min	0.036	0.045	0.054	0.062	0.072
Saturation D_mathrmsat	0.179	0.219	0.282	0.333	0.396
Usable band	0.143	0.174	0.228	0.271	0.324
0.1 dB uses	45%	31%	20%	14%	9%

Table 9. Floor and saturation, dB below the released decoder. The floor is what tiling costs with no early exit at all; saturation is where every tile reaches exit j and the ceiling is attained.

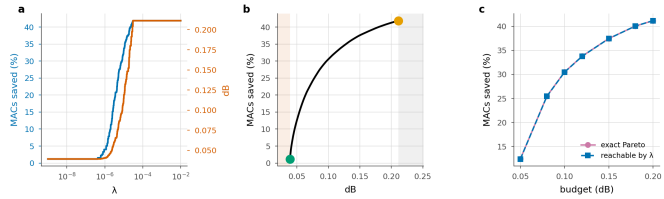

Figure 13. The structure of the allocation, on one frame. **a**, compute and distortion are monotone in the multiplier. **b**, the frontier between the floor (green) and saturation (orange); shaded regions are infeasible and wasted. **c**, what the Lagrangian reaches against the exact Pareto set, enumerated by dynamic programming — the two curves are indistinguishable.

The construction has enough structure to state as propositions, and we verify each numerically rather than asserting it (scripts/verify_theory.py, seven of seven): the allocation decouples per tile; compute is non-increasing and distortion non-decreasing in λ; the sweep traces the lower convex hull and cannot reach an interior point; λ=0 attains the least distortion the ladder can produce; beyond a finite λ, computable in closed form, the allocation is the constant map to exit j at cost exactly c_j; and the ceiling is a function of the split depth alone.

The third has a practical edge. Because the sweep reaches only hull vertices, a budget between two of them is not attainable. We measured what that costs by enumerating the *exact* Pareto set with dynamic programming over tiles — feasible because the cost alphabet has four symbols: the sweep reaches 95 allocations against a Pareto set of 635, and convexity costs at most 0.05 saving points. The standard construction is essentially optimal here.

It is worth saying *why* that loss is small, because the reason is structural and says what would make it smaller. The reachable costs form a lattice whose spacing is set by how much the mean cost moves when the multiplier crosses a switching point: m tiles each move one rung, so the spacing is at most m·max(c_k+1 − c_k)/T saving points, and the convexity loss cannot exceed the largest spacing, because a budget between two levels is served by the lower one. Here m=2 — tiles with identical rows switch together — and the bound is 0.745 points, which the measured spacing attains exactly. Nothing in the expression depends on content or rate, and it falls as 1/T: halving the tile side puts four times as many tiles on the lattice, each carrying a quarter of the step. Fine

granularity is what makes the Lagrangian relaxation lossless in practice, not any property of the images.

Two numbers bound what any budget can do. The **floor** is the distortion of a tiled decode with every tile at full depth — pure tiling penalty, 0.036 dB at q0 rising to 0.072 dB at q63. A budget below it admits no allocation. The **saturation** point is the distortion when every tile takes exit j; any budget at or above it attains the ceiling and a larger one attains nothing more.

A consequence worth stating plainly: at q0 the ceiling is reached at 0.179 dB. The architectural bound is not an asymptote, it is an operating point, and beyond it the limiting factor is the *ladder*, not the budget — an argument for a finer ladder rather than a looser budget. It also identifies a narrow regime: budgets in [0.179, 0.194) dB saturate the lowest rate and nothing else, a window 15 millibels wide.

5.5 Signalled versus predicted allocation

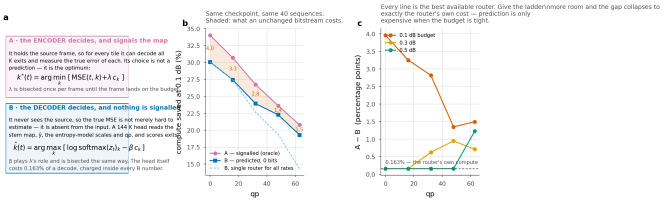

Figure 14. Who decides. (a) A holds the source, so it can decode all K exits per tile and pick the true minimiser; it signals the map at ~79 bits/frame. B never sees the source — a 144 K-parameter head reads ■, the entropy scales and qp — and signals nothing. **(b)** Saving at 0.1 dB; shading is what a bit-exact bitstream costs. **(c)** That cost is rate-dependent and budget-dependent: it collapses to the router's own 0.163% of a decode once the budget is loose enough that both saturate.

	0.1 dB	0.3 dB				
q	A	B	gap	A	B	gap
0	32.3	30.6	**1.7**	41.9	41.7	**0.2**
16	27.6	25.9	**1.7**	41.9	41.7	**0.2**
32	22.5	18.1	**4.4**	41.9	41.7	**0.2**
48	19.8	9.1	**10.7**	40.6	39.5	**1.1**
63	16.8	3.5	**13.3**	38.4	36.3	**2.1**

Table 10. Signalled against predicted at two budgets, same checkpoint and test set, with one router trained against the deployed oracle at a single λ.

Configuration A is exact by construction and costs bits; B is approximate and costs none. The comparison splits in two.

At the rates near the λ the router was trained at, **prediction is nearly free**: 30.6% against 32.3% at q0 and 25.9 against 27.6 at q16 — a gap of 1.7 points for zero added bits and a byte-identical file.

Away from it the gap grows to 13.3 points, and it tracks |β|, the tilt the bisection has to apply to drag a router trained at one operating point to another. A large tilt lets the cost term dominate the logits and discards the content ranking the router learned. The remedy is not a better architecture but a router per operating point, which a deployment would have anyway: one set of weights serves all rates, and a 144K head per rate is 0.3% of the model each. We report the single-router number because it is the honest one for a system that trains once, and note that it understates what B can do.

Loosening the budget closes it from the other side. At 0.3 dB the gap is 0.2 points at the three lowest rates — exactly the router's own 0.163% of decode, i.e. its *prediction* is then free — and at most 2.1 at the highest. Once the budget saturates the ladder both configurations send every tile to the same rung and nothing is left to predict wrongly. Configuration B is expensive in exactly one regime: a tight budget at a rate far from the router's training point.

5.6 Signalling only what the router gets wrong

A and B are the two ends of a single scale, not two designs. The encoder can run the decoder's router — it reads only decoded data — so it knows, tile by tile, where the prediction will be wrong. Nothing forces it to correct all of them.

Configuration C signals a fraction ρ of the tiles and leaves the rest to the router. The overrides are chosen by Lagrangian regret, which is exactly what the objective loses on that tile by staying silent; β is held at B's value and λ is bisected over the overrides to land back on the budget. The map costs an entropy-coded mask, N·H■(ρ) bits, plus 3 per override.

[hybrid.png missing]

q	B	5%	10%	20%	50%	A
bits/frame	0	14	26	44	83	100
0	30.6	31.2	31.7	32.1	32.3	32.3
16	25.9	26.2	26.5	26.9	27.4	27.6
32	18.1	19.0	19.9	20.7	22.1	22.5
48	9.1	10.6	12.2	14.4	18.5	19.8
63	3.5	5.1	6.9	9.2	14.1	16.8

Table 11. Configuration C at 0.1 dB. Columns are the fraction of tiles the encoder overrides; the two end columns reproduce B and A to within 0.1 points, which is the check that the interpolation is real.

The two ends check out. At ρ=0 the measurement reproduces configuration B to the second decimal at every rate, and at ρ=1 it reproduces A. Neither is imposed — both fall out of the same code path — so the columns between them are measuring something real.

Most of the gap is cheap. At q0, overriding a tenth of the tiles for 26 bits per frame recovers 61% of the gap; a fifth recovers 83%. At q63, where the gap is widest, the same fifth of the map — 44 bits against 100 for the full map — recovers 42%, and half recovers 80%. Recovery is concave everywhere, so the first bits spent are always the most useful ones.

Why it is concave, and what bounds it. At a fixed λ the objective is separable, so overriding a set removes *exactly* the sum of that set's regrets: the recovery of the *objective* is the Lorenz curve of the per-tile regret distribution, and its curvature is that distribution's Gini coefficient, 0.57–0.88 across rates. What the table reports is not objective but saving at a fixed distortion, and returning to the budget means re-bisecting λ, which converts the recovered distortion into compute at the local exchange rate rather than one for one. The Lorenz curve is therefore an upper bound, and the measurement says so: every point in panel c lies on or below the diagonal, reaching 37–100% of the bound. The concentration of the router's mistakes is what makes partial signalling worth doing; the re-tuning is what stops it from being free.

Configuration C is what we would ship where a small map is tolerable and a large one is not. It also reframes the A–B gap: it is not the price of prediction, it is the price of *silence*, and silence is priced per tile.

5.7 Does the map have to be recomputed?

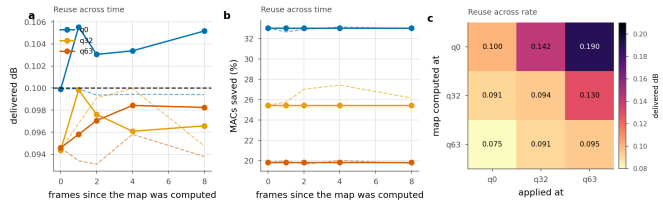

Figure 16. Reusing an exit map. Solid is the transferred map, dashed the one recomputed in place. **a**, **b**: reuse across frames of the same sequence. **c**: reuse across quality index; the entry is the delivered distortion against a 0.1 dB budget.

Configuration A costs the encoder about one extra decode per frame. Whether that matters depends on how often the map has to be recomputed, which nobody has checked.

Across time it barely has to be. Frame 0's map applied eight frames later costs 0.005 dB at q0 — five percent of the budget — and no saving at all: 33.05% transferred against 32.66–33.15% recomputed. The allocation is a property of where the content is hard, and that moves slowly. A codec would search once per group of pictures and divide the encoder cost by the group length.

Across rate it very much does, and the direction matters. The map found at q0 applied at q63 delivers 0.190 dB against a 0.1 dB budget: it claims the low-rate saving of 33.05% while spending nearly twice the quality it is allowed. The reverse is safe but wasteful — the q63 map at q0 delivers 0.075 dB and only 19.8% where 33.1% was available. Shallow exits are cheap in quality at low rate and expensive at high rate, so a map is calibrated to the rate it was found at, and reusing it upward silently breaks the quality guarantee. Search once per rate, reuse across frames.

5.8 Complexity and wall-clock

q	Released (ms)	Routed sorted (ms)	Saved (%) measured	MACs
0	111	78	**29.1**	35.3
32	111	86	**22.4**	28.0
63	111	94	**15.6**	20.1

Table 12. Wall-clock, 1080p, median of 40 interleaved iterations, at the 0.1 dB operating point. “MACs” is what the arithmetic predicts; “measured” is the sorted per-tile loop.

A saving in multiply-accumulates is not a saving in time. Tiling itself costs 3.6% before anything exits early — the deepest-exit tiled decode against the released full-frame one, same arithmetic, same weights — and the routed decode realises 27.9% at q0 against the 35.3% its operations predict. Four fifths of the predicted saving arrives; the missing fifth is the tiling overhead plus the bookkeeping of a shrinking active set, which a MAC count cannot see. The shortfall is proportional rather than constant: 15.6% realised against 20.1% at q63.

Sorting the tiles once by exit depth recovers part of it. The obvious implementation of a shrinking active set — a boolean mask, a gather of survivors and a scatter of the finished at every group boundary — forces a device-to-host synchronisation per boundary. Descending by exit depth, “still active at group g” becomes a contiguous prefix instead: each group is a slice rather than a gather, the boundaries come from one cumulative count rather than a mask per group, and the finished tiles return through the inverse permutation in a single scatter. The arithmetic is unchanged and the output is bit-identical on GPU. The saving at q0 goes from 27.9% to 29.1% — 1.2 points, a fifth of the shortfall, for a change that touches no arithmetic.

The same lesson applies to our own accounting. Every configuration B number in this paper charges the router at its share of the decoder's multiply-accumulates, 0.163%. Timed on the padded 2048×1280 frame the decoder actually sees, interleaved against a deepest-exit decode so a co-tenant's load lands on both equally, the head costs 0.50% — 3.0× what its arithmetic predicts, and for the same reason as everything else in this section: a 144 K-parameter head is launch overhead, and a MAC count cannot see a launch. Charged at measured time rather than at operations, every B number in this paper would fall by a further 0.33 points. We leave them charged at MACs because that is the convention the rest of the literature reports in, and record the correction here rather than letting it sit unstated.

We report the shortfall because a paper that quotes only operations would report 35.3% where the same code, run as written, delivers 29.1%. The direction is the one that matters: operations are an *optimistic* bound on this method, and the optimism grows with how much of the frame exits early.

5.9 The right ladder depends on the budget

Ladder	Config	Ceiling	0.1 dB	0.3 dB	0.5 dB
RECIPE512	K6 j2 256px	41.9	23.8	41.0	41.9
BEST	K6 j2 256px	41.9	**24.2**	38.5	41.3
BEST128	K6 j2 128px	41.9	19.3	36.6	40.6
FINE12	K12 j4 128px	50.3	19.0*	**42.0**	**48.6**

Table 13. Ladder configurations, mean saving (%) over the five rates, same test set and protocol. * one rate is infeasible at that budget — the ladder's floor exceeds it — so the mean is over the remaining four.

Section 5.4 argued that once a budget saturates a ladder the only way to spend more is a rung that does not exist. The table measures it. A finer ladder (K=12, j=4) has a ceiling of 50.3% against 41.9%, and the orderings cross between 0.1 and 0.3 dB. At 0.1 dB the coarse ladder wins, and the fine one cannot even reach the budget at the highest rate: splitting later puts twice as many blocks in the per-tile section, which raises the floor. At 0.5 dB the fine ladder wins by 6.7 points, 48.6% against 41.9%, because the coarse one has been pinned at its ceiling since 0.3 dB.

So the ladder is not a hyperparameter to be tuned once. It is a function of the operating point, and the floor-saturation window is what tells you which side of the crossover you are on.

6. Limitations

The seam is reduced, and the exact remedy is not usable as it stands. Canvas coupling removes 47–91% of the floor and is bit-exact at uniform depth, yet collapses the routed saving because routing puts neighbouring tiles at different depths. Whether a decoder trained with coupling recovers both at once is open, and it is the experiment we would run next. Until it exists the floor is a cost this method pays.

Intra frames only. This is the image path of a video codec. Extending the ladder to inter frames raises a question this paper does not answer: an exit map propagates through the reference chain, so a shallow tile in one frame is a worse reference for the next.

Training is not converged. The measured saving is still increasing at every checkpoint measured more than once, so these numbers are a lower bound.

A single decoder. All results are on DCVC-UF's intra decoder. Nothing in the method is specific to it — the ladder needs only a residual trunk with a shared head — but that is an argument, not a measurement.

7. Conclusion

Learned decoders spend a constant amount of computation on a non-constant world. An early-exit ladder over the tiles of a frame recovers a substantial fraction of it — 32.3% to 16.8% of decoder MACs for a 0.1 dB budget, at a BD-Rate cost of 0.88% — without touching the encoder or the coded payload.

Three lessons we would carry to any spatially adaptive decoder, none of them about early exit. **Tiling is the dominant cost and it is governed by depth.** Its entire cause is a single 3×3 that is 0.29% of the arithmetic, the penalty grows as the square of how many such convolutions run per tile, and the corrupted-area fraction that is usually quoted predicts it badly. **The exact remedy is in tension with the thing it enables.** Giving each convolution its real neighbour is bit-identical at uniform depth and collapses the allocation under routing, because routing is the deliberate violation of the condition that makes it exact — a trap that any method combining spatial adaptivity with tiled inference will walk into. **A quality budget is only a control variable inside a measurable window.** Below the floor it admits nothing, above saturation it buys nothing, and reporting a saving without saying where in that window it

sits leaves out the most useful part of the result.

And three cautions about measuring any of this. A saving in operations is an optimistic bound on a saving in time, and the optimism scales with the saving. A learned router should be compared against a free one: ranking tiles by the bits already spent on them costs nothing and gets most of the way. And a timing harness will report numbers whether or not it is timing the right device — this one did, for months, and the tell was a decoder that appeared to slow down with the quality index.

References

[1] S. Wang et al. Adaptive patch exiting for scalable single image super-resolution. ECCV, 2022.

[2] J. Ballé et al. Variational image compression with a scale hyperprior. ICLR, 2018.

[3] G. Huang et al. Multi-scale dense networks for resource efficient image classification. ICLR, 2018.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD curves. VCEG-M33, 2001.

[5] B. Bross et al. Overview of the Versatile Video Coding standard. IEEE TCSVT, 2021.

[6] Z. Cheng et al. Learned image compression with discretized Gaussian mixture likelihoods. CVPR, 2020.

[7] W. Fedus et al. Switch Transformers. JMLR, 2022.

[8] G. Huang et al. Glance and Focus networks for dynamic visual recognition. IEEE TPAMI, 2023.

[9] G. J. Sullivan et al. Overview of the High Efficiency Video Coding standard. IEEE TCSVT, 2012.

[10] E. Jang et al. Categorical reparameterization with Gumbel-softmax. ICLR, 2017.

[11] T. Kaseva et al. Per-channel autoregressive linear prediction padding in tiled CNN processing of 2D spatial data. arXiv:2502.12300, 2025.

[12] X. Kong et al. ClassSR: a general framework to accelerate super-resolution networks by data characteristic. CVPR, 2021.

[13] J. Li, B. Li, Y. Lu. Neural video compression with feature modulation. CVPR, 2024.

[14] J. Li, B. Li, Y. Lu. Uniformly accelerated neural video codec with feature refinement. ACM MM, 2024.

[15] Y. Kaya et al. Shallow-deep networks: understanding and mitigating network overthinking. ICML, 2019.

[16] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.

[17] M. Phuong, C. H. Lampert. Distillation-based training for multi-exit architectures. ICCV, 2019.

[18] S. Scardapane et al. Why should we add early exits to neural networks? Cognitive Computation, 2020.

[19] W. Sun et al. Multi-exit self-distillation with appropriate teachers. FITEE, 2024.

[20] S. Teerapittayanon et al. BranchyNet: fast inference via early exiting from deep neural networks. ICPR, 2016.

[21] B. Bross et al. Versatile Video Coding. IEEE TCSVT, 2021.

[22] F. Yang et al. Slimmable compressive autoencoders for practical neural image compression. CVPR, 2021.

[23] M. A. Y■lmaz et al. Slimmable video codec. CVPRW, 2022.

[24] A. Kuznetsova et al. The Open Images Dataset V4. IJCV, 2020.

[25] A. Mercat et al. UVG dataset: 50/120fps 4K sequences for video codec analysis. ACM MMSys, 2020.

[26] H. Wang et al. MCL-JCV: a JND-based H.264/AVC video quality assessment dataset. ICIP, 2016.

[27] T. Blard et al. Spatial competition for low-complexity learned image compression. arXiv:2605.13243, 2026.

[28] Y. Shoham, A. Gersho. Efficient bit allocation for an arbitrary set of quantizers. IEEE TASSP, 1988.

[29] A. Ortega, K. Ramchandran. Rate-distortion methods for image and video compression. IEEE SPM, 1998.

[30] Adaptive test-time compute allocation for reasoning LLMs via constrained policy optimization. arXiv:2604.14853, 2026.

[31] H. A. Alhaija et al. Local padding in patch-based GANs for seamless infinite-sized texture synthesis. arXiv:2309.02340, 2023.

[32] C. Innamorati et al. Overlap training to mitigate inconsistencies caused by image tiling in CNNs. arXiv:1812.02203, 2018.

[33] Z. Jia et al. Towards practical real-time neural video compression. CVPR, 2025.

[34] B. A. Hasan et al. Adaptive inference: theoretical limits and unexplored opportunities. arXiv:2402.04359, 2024.

[35] Y. Gao et al. Exploring the rate-distortion-complexity optimization in neural image compression. CVIU, 2024.

[36] Y.-H. Ho et al. On the rate-distortion-complexity trade-offs of neural video coding. arXiv:2410.03898, 2024.

[37] C. Zhang, W. Gao. Learned rate control for frame-level adaptive neural video compression via dynamic neural network. arXiv:2508.20709, 2025.

[38] Y. Geifman, R. El-Yaniv. Selective classification for deep neural networks. NeurIPS, 2017.

[39] D. Madras et al. Predict responsibly: improving fairness and accuracy by learning to defer. NeurIPS, 2018.

[40] H. Mozannar, D. Sontag. Consistent estimators for learning to defer to an expert. ICML, 2020.

[41] G. DeSalvo et al. Budgeted multiple-expert deferral. arXiv:2510.26706, 2025.